The Neuro-Wheel

MECH 307

Group 21

12/12/2025

[redacted], [redacted], Roger Perkins, [redacted]

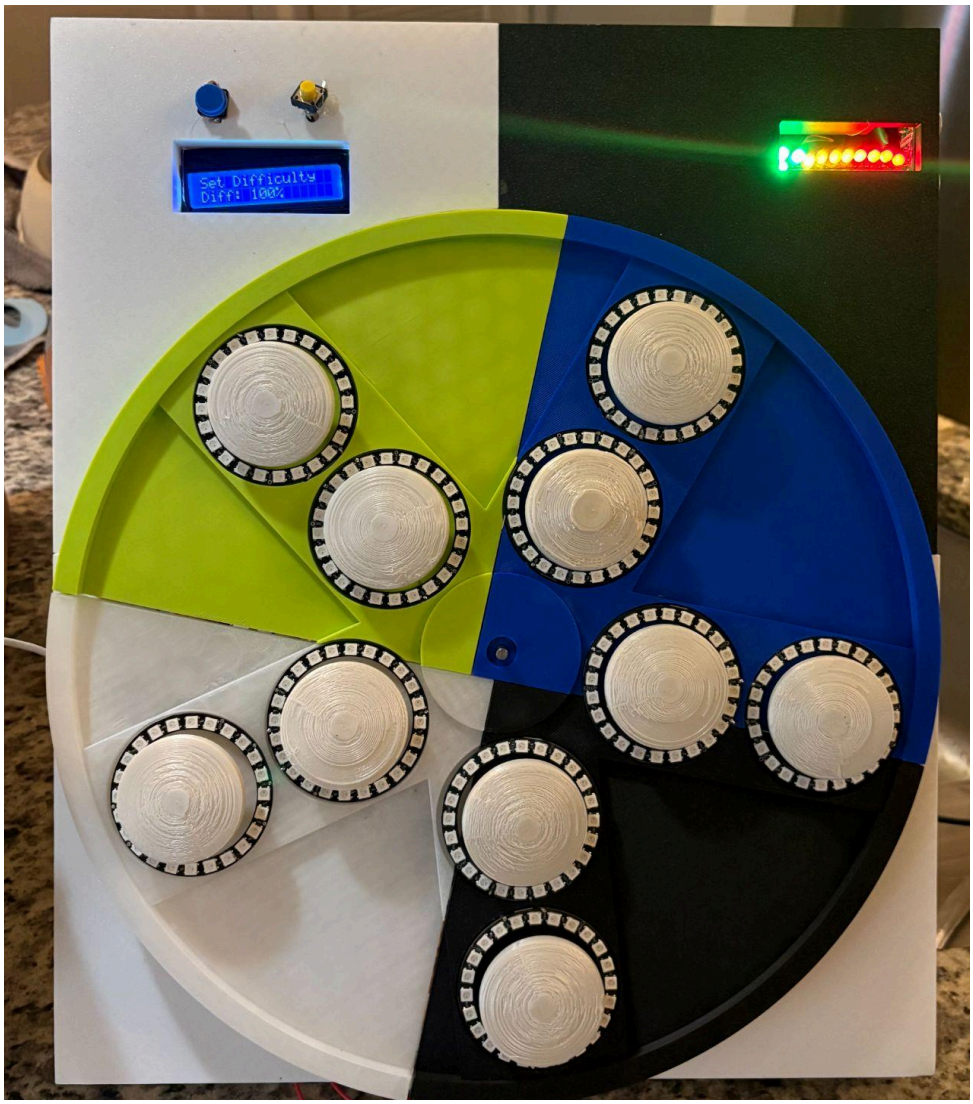


Table of Contents

Design Summary 2
System Details 3
- **Designs and Drawings** 3
- **Labeled Pictures of Internal Components** 5
- **Software Flowcharts** 7

Scheduling 8
Design Evaluation 8
Partial Parts List 10
Lessons Learned 11
Appendix 13

Design Summary

The Neuro-Wheel is a fun, Simon Says-type game, designed to test the reaction time of multiple players. The device consists of a base (1) and wheel (2) with 10 lights (3) and buttons (4). The game can be operated at varying difficulties, selected by placing the hand at a distance from the difficulty sensor, with the difficulty displayed by the colored LEDs (5). Harder difficulties have a quicker, more punishing reaction time. Once the play button (6) is selected, music begins playing, and the wheel spins, with lights illuminating randomly, one at a time, for 20 seconds. It is a 2-player game, with the winner being the player able to collect the highest score during their round. The winner and their score is shown on the LCD display (7), and the game can then be restarted.

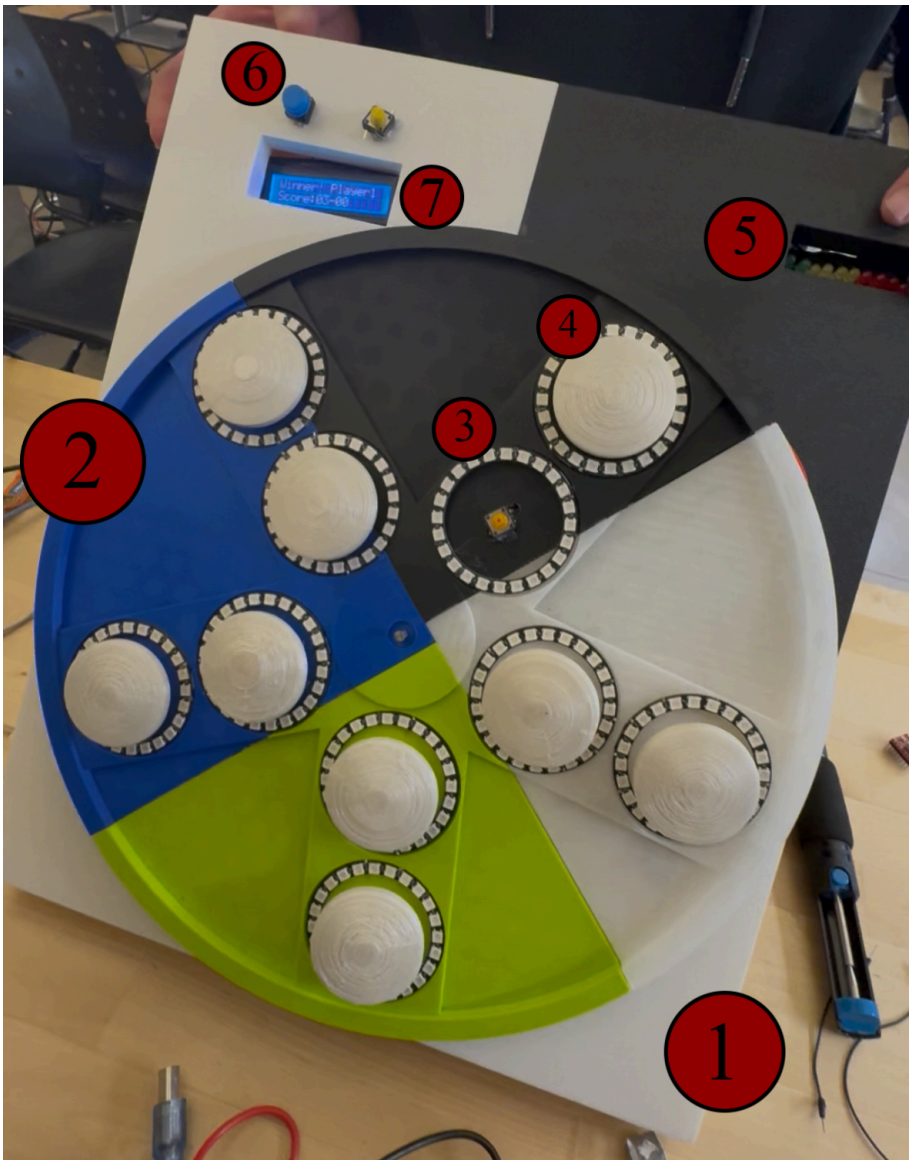


Figure 1: Labeled image of the Neuro-Wheel.

System Details:

Designs and Drawings

Below are drawings of the concept for the game, with the first figure showing what the entire board was first conceived to look like and the following figure describing each component and its purpose.

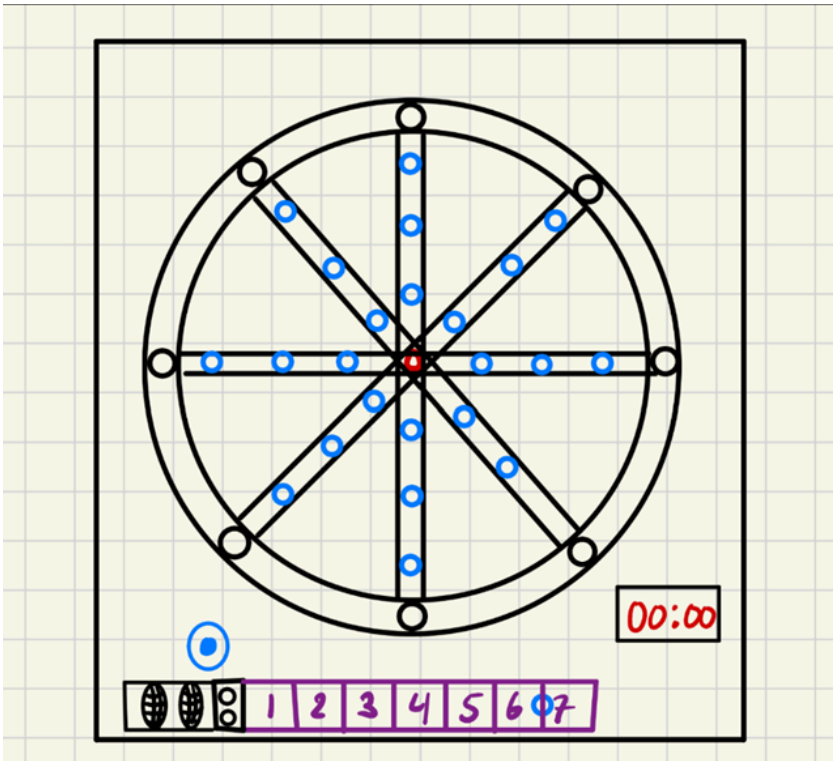


Figure 2: Full drawing depicting the initial product design.

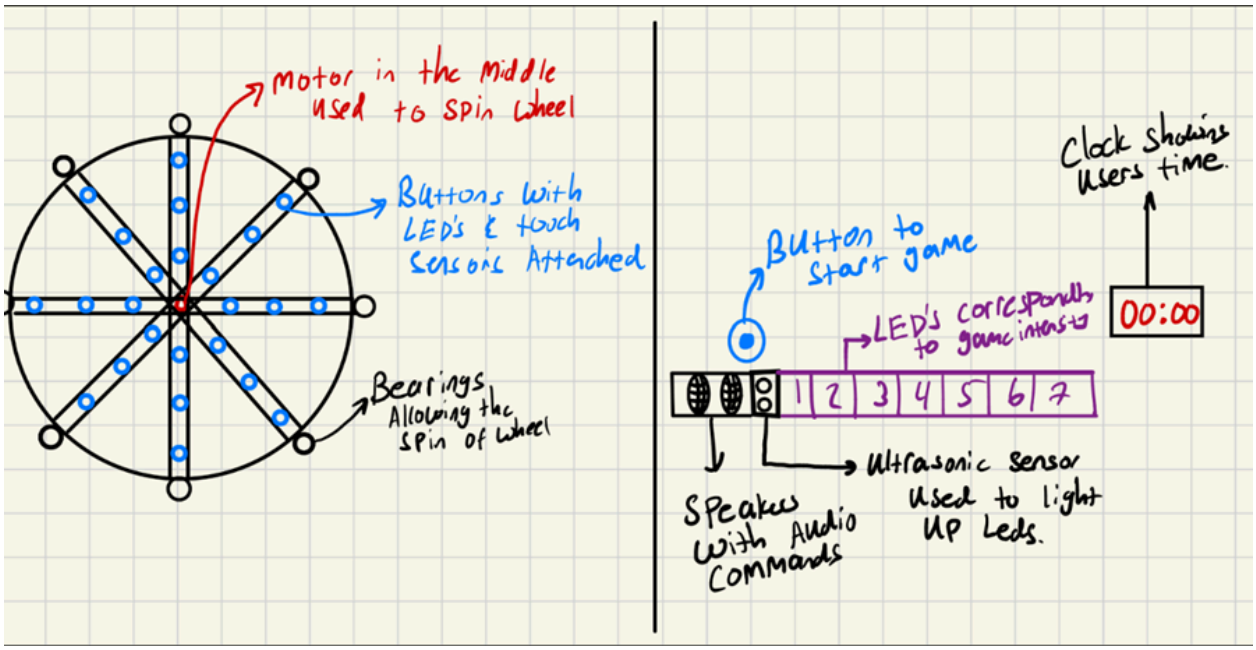


Figure 3: Dissected drawings labeling each part from Figure 3 (above) and its purpose.

Figures 2 and 3 were the initial drawings of what the model was going to look like. In the end the final product didn't deviate far from the initial idea. Additionally, a functional diagram composed of pictures of each component has been made below to aid in understanding the construction of the final product.

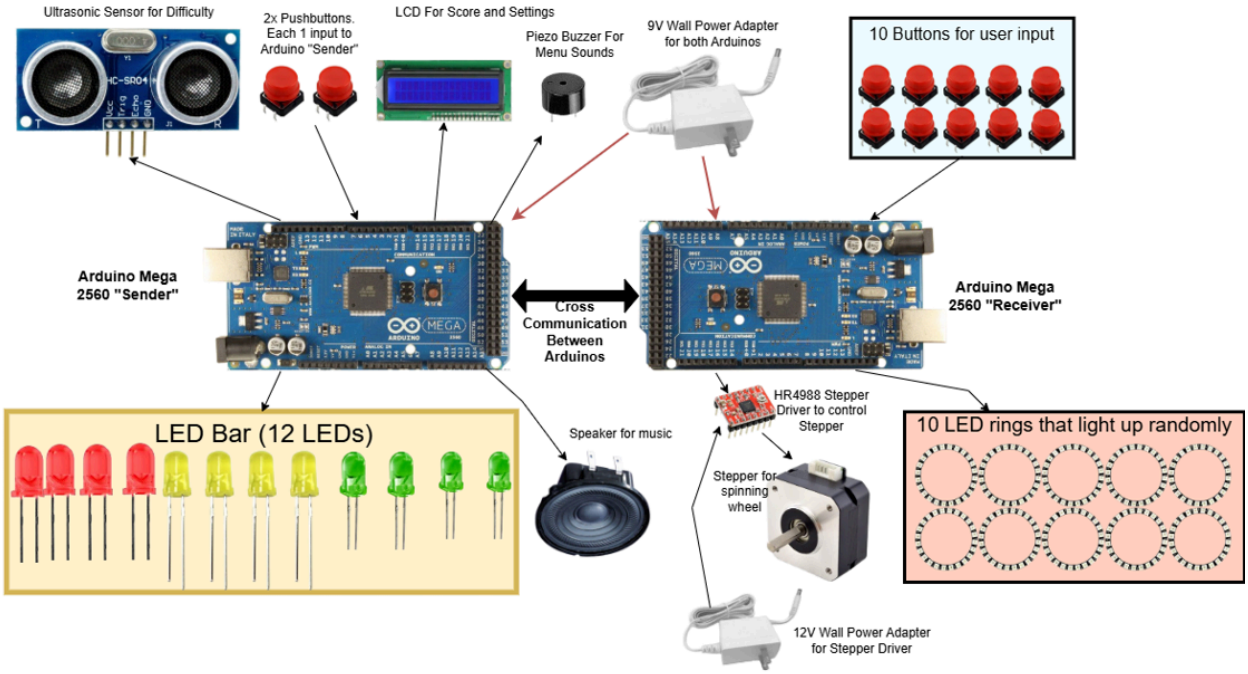


Figure 4. Functional Diagram showing how each component connects to the rest of the system.

Labeled Pictures of Internal Components

In addition to Figure 1 showing the front components, the following two figures are labeled pictures of the components used internally within the game's design.

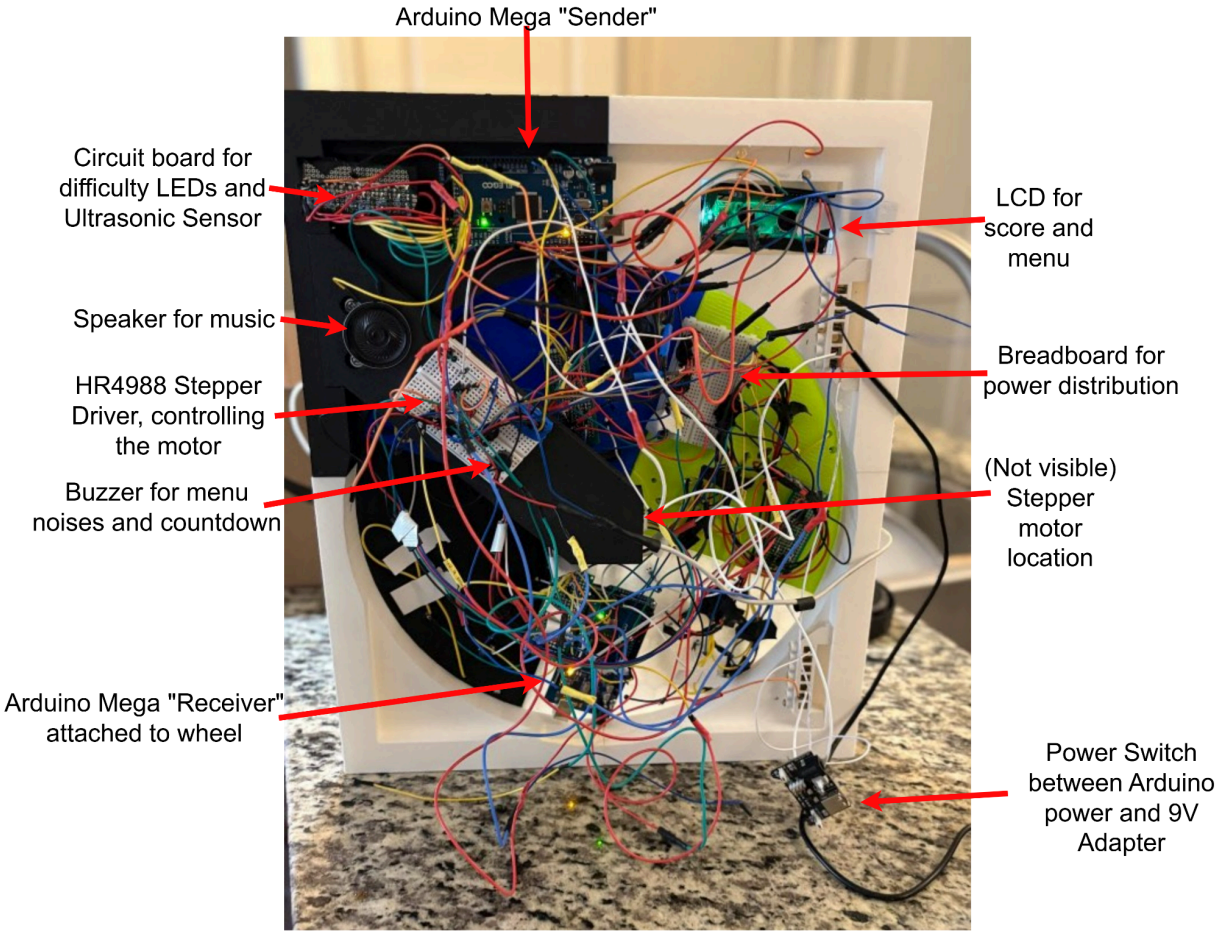


Figure 5: Labeled picture of all underside components of the game board.

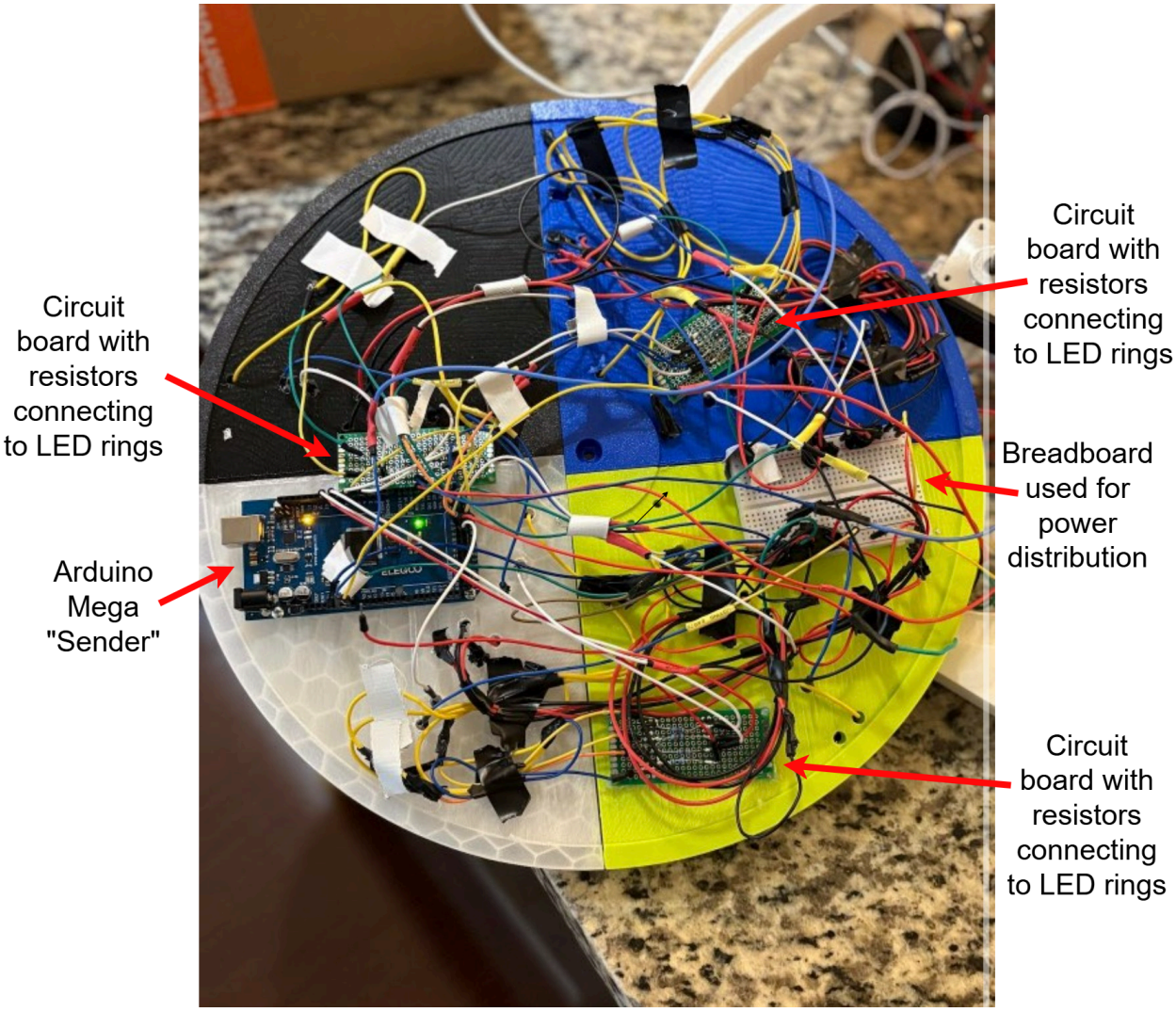


Figure 6: Labeled underside of the game wheel for clarity.

Software Flowcharts

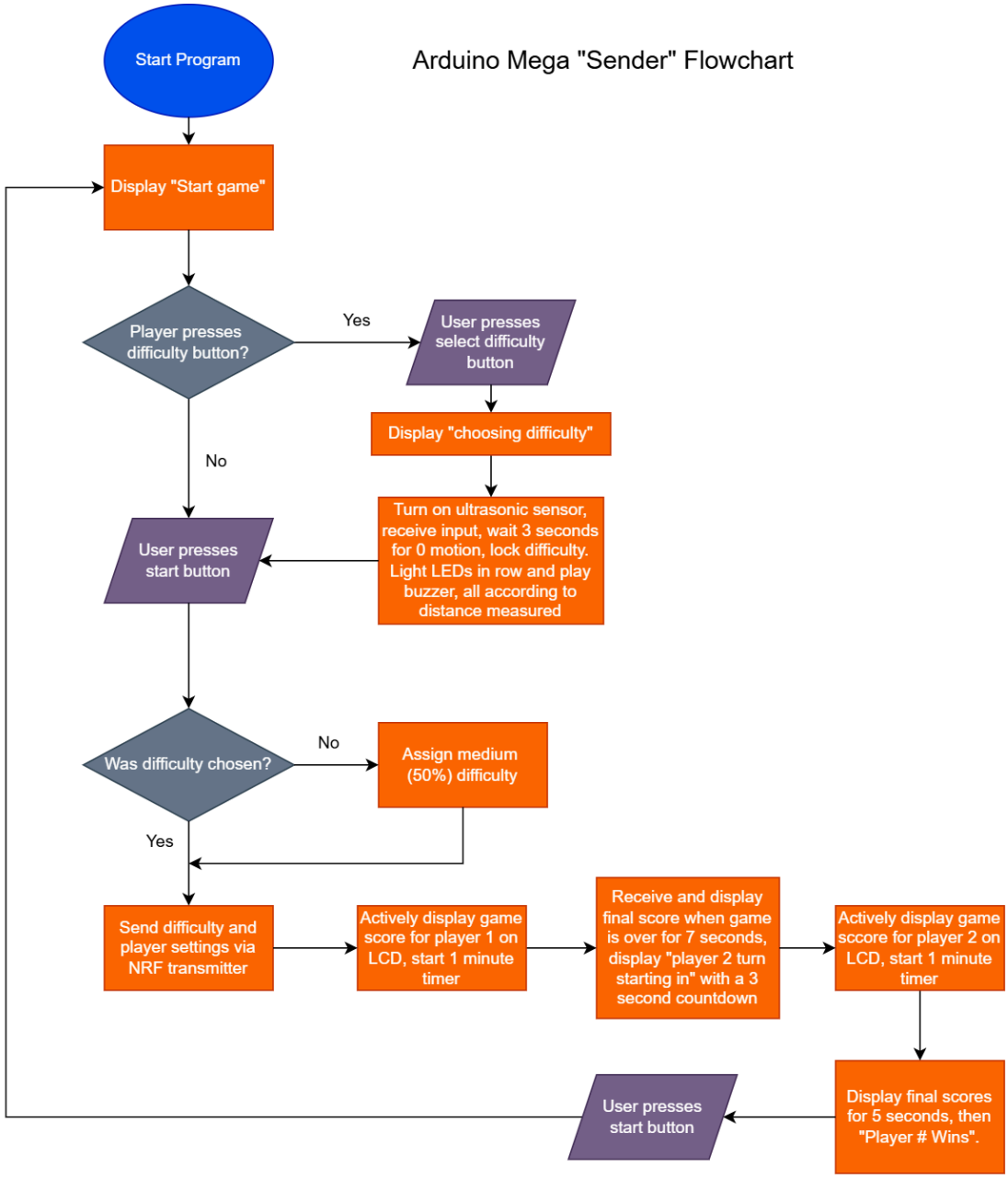


Figure 7: Software flowchart for the “sender” arduino.

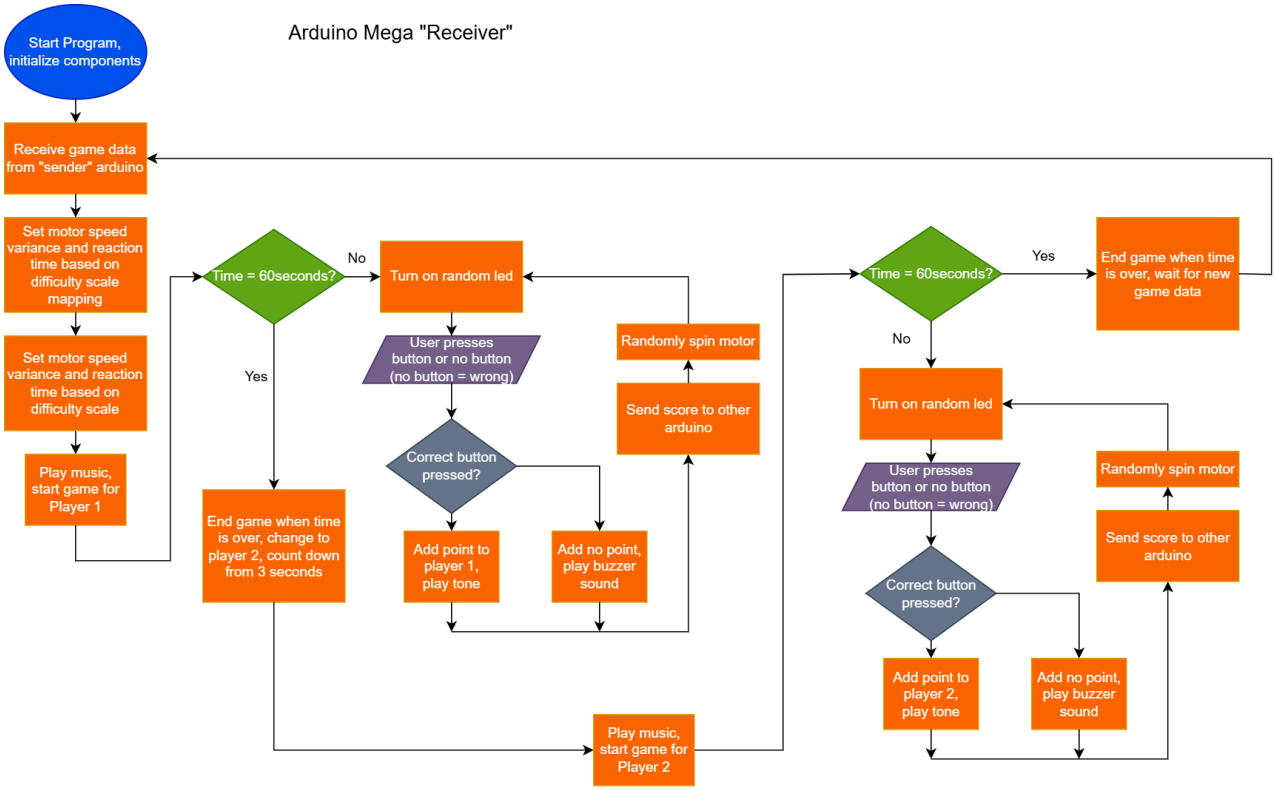


Figure 8: Software flowchart for the "receiver" arduino.

Figures 7 and 8 summarize the software for the two-controller system: the Sender Arduino manages the user interface and game sequencing (difficulty selection, LCD/LED feedback, countdown/audio cues, and round timing), while the Receiver Arduino handles gameplay execution (LED ring targets, button scoring, stepper motor control, and sending score updates back to the sender).

Scheduling

The group originally planned to meet every Thursday, after 4pm, for at least 30 minutes to discuss, plan, and build the project. Additional weekend meetings or changes to meetings, along with general project info, were communicated via a text message group chat. Within the final four weeks of the project, the group agreed to meet two to three times weekly in order to speed up the project's completion and reduce daily meeting times. As the project due date drew near, the group spent the last two weeks printing parts and combining all components multiple days in a row to make sure the project was perfect before presentations.

Design Evaluation

The Neuro-Wheel successfully demonstrated functionality in four of the six required functional element categories at the time of the final demonstration: **Output Display** (LCD and LEDs), **Audio Output** (buzzer-style sound cues), **Manual User Input** (pushbuttons), and **Logic,**

Processing, and Control (programmed state-based control using two Arduino Megas). The LCD correctly displayed the selected difficulty, countdown prompts, and player scores at the appropriate times, and the LEDs provided clear visual feedback for gameplay and difficulty indication. All 12 buttons functioned as intended, including start/stop control, difficulty selection, and the game input buttons used to score points during each round. The overall game behavior: timing, state transitions, score tracking, and player sequencing, was handled entirely in software.

The system used **two Arduino Megas** to distribute tasks: one managed the gameplay LEDs and button scoring, while the other managed the user interface features (LCD and difficulty indicator LEDs), sensor input, and overall game sequencing. Communication between the controllers was implemented through a wired serial link using the RX/TX pins, which provided stable and consistent performance during the demonstration.

Two additional categories were included in the final design, but did not function as intended at demonstration. The **Automatic Sensor** category was intended to be satisfied using an ultrasonic distance sensor to set difficulty based on hand position. This feature worked reliably during development and consistently mapped distance readings to difficulty levels which can be seen in Figure 9 below. The **Actuators, Mechanisms, and Hardware** category was addressed using a stepper motor intended to rotate the wheel during gameplay. The motor operated electrically and produced the expected motion profile.

Finally, the communication method was revised late in the build. The original concept used RF transceivers to avoid wire twisting from wheel rotation. While the RF link worked during early testing, it became unreliable over time and was ultimately replaced with a wired serial connection to ensure a complete, stable system for the demonstration. Similarly, the DFPlayer Mini–based audio system experienced repeated module failures during integration; therefore, the audio output was simplified and the speaker was used only for buzzer-style cues rather than music playback.

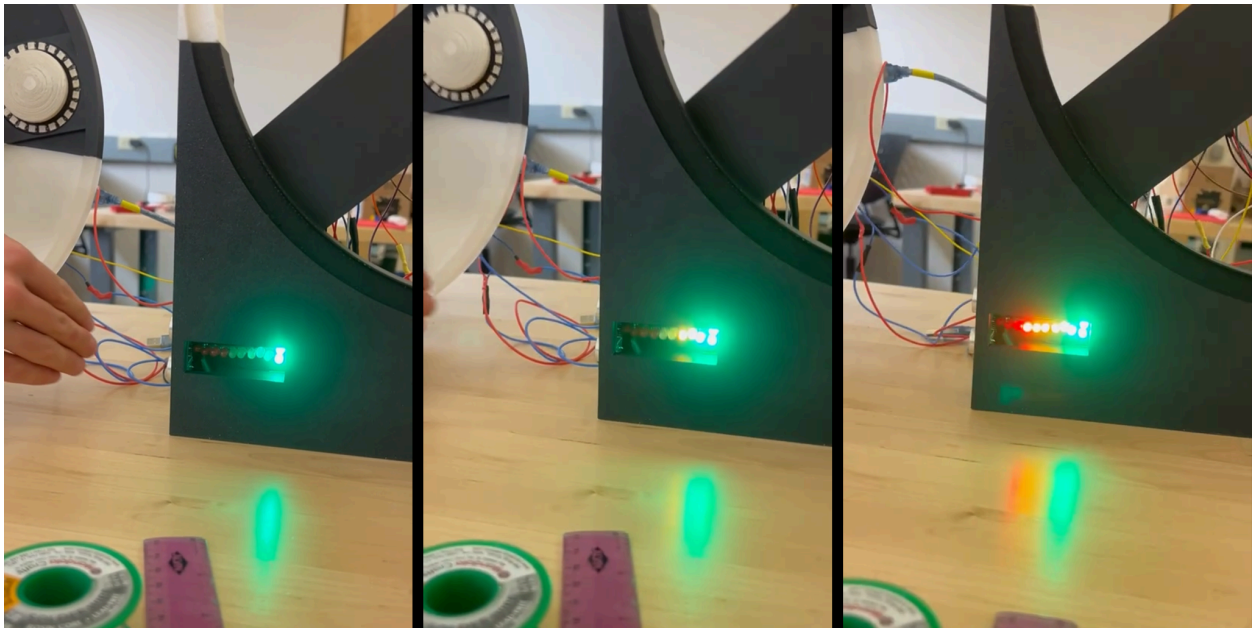


Figure 9: Ultrasonic sensor being used to display various difficulties

Partial Parts List

Parts/Materials	Quantity	Price Per Unit	Total cost (USD)	Supplier
LED Rings	2 sets of 5 rings	23.99	47.98	Amazon
Game Pushbuttons	10	1.346	13.46	DigiKey
Speaker	1	4.18	4.18	DigiKey
DFPlayer Mini	1	5.9	5.9	DigiKey
12V wall adapter	1	3.39	3.39	Amazon
TwoTrees Nema17 Stepper	1	0	0	[redacted]
Power Module		0	0	Provided Arduino Mega Kit
Universal Prototype Board	4	0	0	[redacted]
PLA	891.16 grams	0	0	I2P Lab

Table 1: Partial parts list including part description, model number, vendor/source, and unit price.

Table 1 summarizes the major non-standard components used to build the Neuro-Wheel and their estimated costs. Most of the project's out-of-pocket cost came from the LED rings and the game pushbuttons, which were purchased externally (Amazon and DigiKey). Several other required items, such as the Arduino Mega boards, prototype boards, stepper motor, and PLA,

were provided through the MECH 307 kit, [redacted], or the I2P Lab and are therefore listed as $0. Prices shown reflect the unit cost at the time of purchase (or the closest available equivalent when substitutions were required).

Lessons Learned

- **3D printing (segmented base + button cap finish)**
  Problem: When the base was split into segments using the PrusaSlicer connector feature, reprinting a single damaged corner was difficult because connector placement had to be re-selected each time, making the joints hard to reproduce consistently. In addition, support removal on the concave button caps often tore the top surface and left a rough, stringy finish.
  Solution: Base segments were reprinted as needed and assembled using adhesive when small connector misalignments occurred. For the button caps, tree supports improved results in some cases, but the most reliable fix was light sanding and cleanup after printing.
  Recommendation: If partial reprints are likely, avoid "on-the-fly" connector placement—design alignment features directly in CAD (tabs/dowels/keyways) or document connector settings carefully to ensure repeatability. For small cosmetic parts, use support strategies that minimize surface contact (tree supports, interface settings), and plan time for simple post-processing (sanding).

- **Time management and integration planning**
  Problem: Early in the project, weekly 30-minute meetings were not sufficient for coordinating tasks and integrating hardware/software, especially with exams and conflicting schedules. This compressed testing and iteration into the final weeks.
  Solution: The team increased meeting frequency and duration near the end to complete assembly and debugging.
  Recommendation: Schedule longer integration sessions earlier (even one 1-2 hour meeting per week) and set internal deadlines for "first full system integration" well before the final demo to allow time for reprints, wiring revisions, and software tuning.

- **Mechanical/electrical compatibility and troubleshooting discipline**
  Problem: Several issues occurred only after full assembly. For example, the stepper system worked during early testing but later failed due to a simple missing driver connection.
  Solution: The electrical issue was resolved by re-checking the wiring and reconnecting the missing driver wire.
  Recommendation: Don't skip the basics. Use a wiring checklist and continuity checks whenever behavior changes after assembly.

- **System reliability and contingency planning (RF communication)**
  Problem: RF transceivers worked during early testing but became unreliable later, forcing a last-minute change in the communication approach.
  Solution: The team switched to a direct wired serial RX/TX connection to ensure stable communication for the demonstration.
  Recommendation: Build a "Plan B" early for any high-risk subsystem. If wireless communication is optional, validate it early and keep a wired fallback ready so the overall device remains functional even if a module becomes unreliable.

- **System Integration of Code**
  Problem: While several subsystems were achievable on their own (ultrasonic difficulty setting, LCD prompts, scorekeeping, LED/motor behavior, etc.), integrating them into one reliable game was much more difficult than expected. Combining features required additional variables, shared timing logic, and careful ordering of operations so the game progressed correctly from difficulty selection → prompts/countdown → gameplay → scoring → next player. Maintaining this sequence and preventing conflicts between functions became the most time-consuming part of development.
  Solution: We began integrating earlier and combined features one phase at a time in the same order the user experiences them. After each integration step, we tested and debugged before adding the next feature, which helped isolate problems and keep the game flow consistent and user-friendly.
  Recommendation: Avoid waiting until the end to merge code. Build the program in phases and integrate one working feature at a time into the main loop/state machine. Testing after each phase prevents "everything broke at once" problems and leads to smoother final integration.

Appendix A - Wiring Schematics

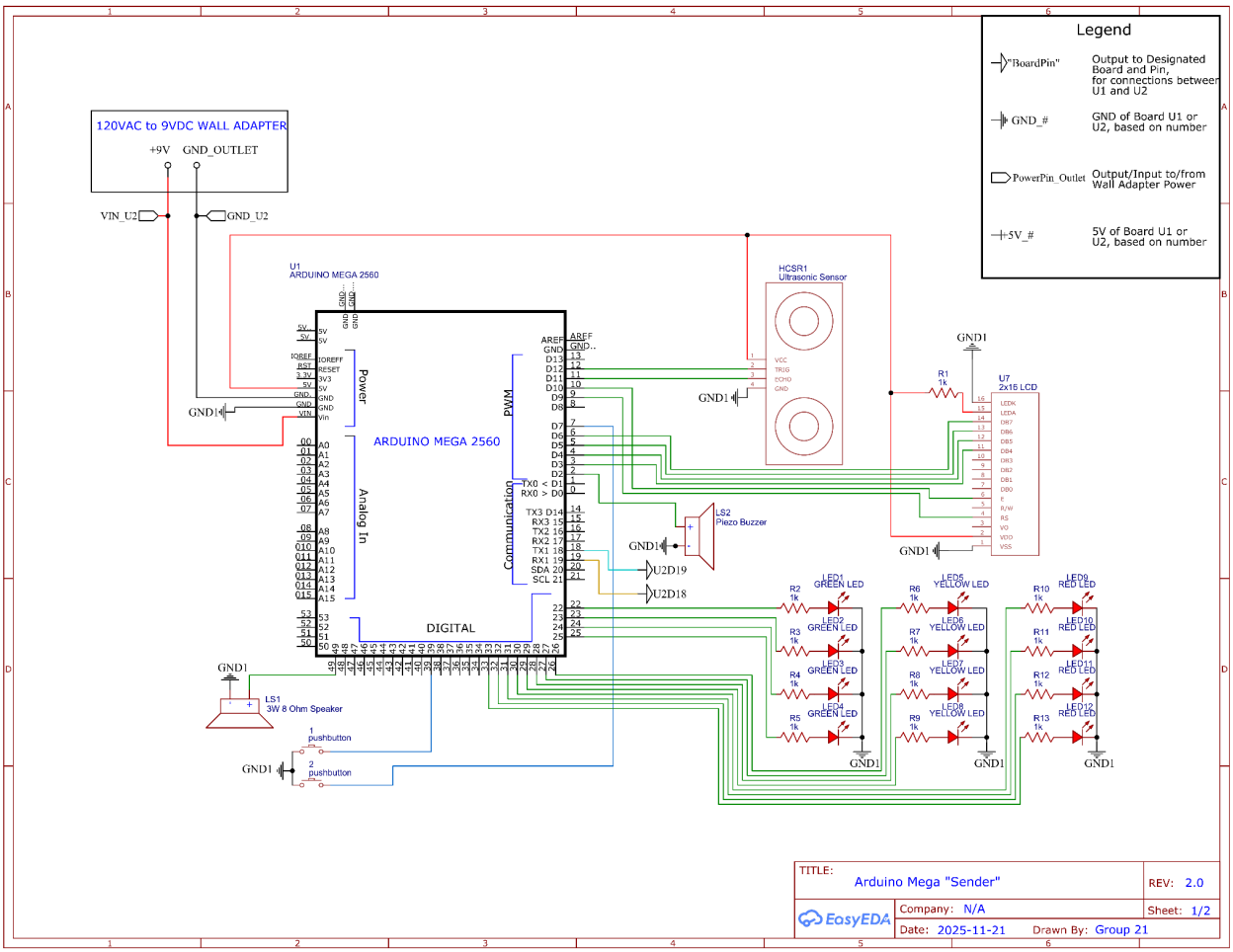


Figure 10: Wiring schematic for the Arduino Mega used for setting difficulty and starting the game, nicknamed “Sender”.

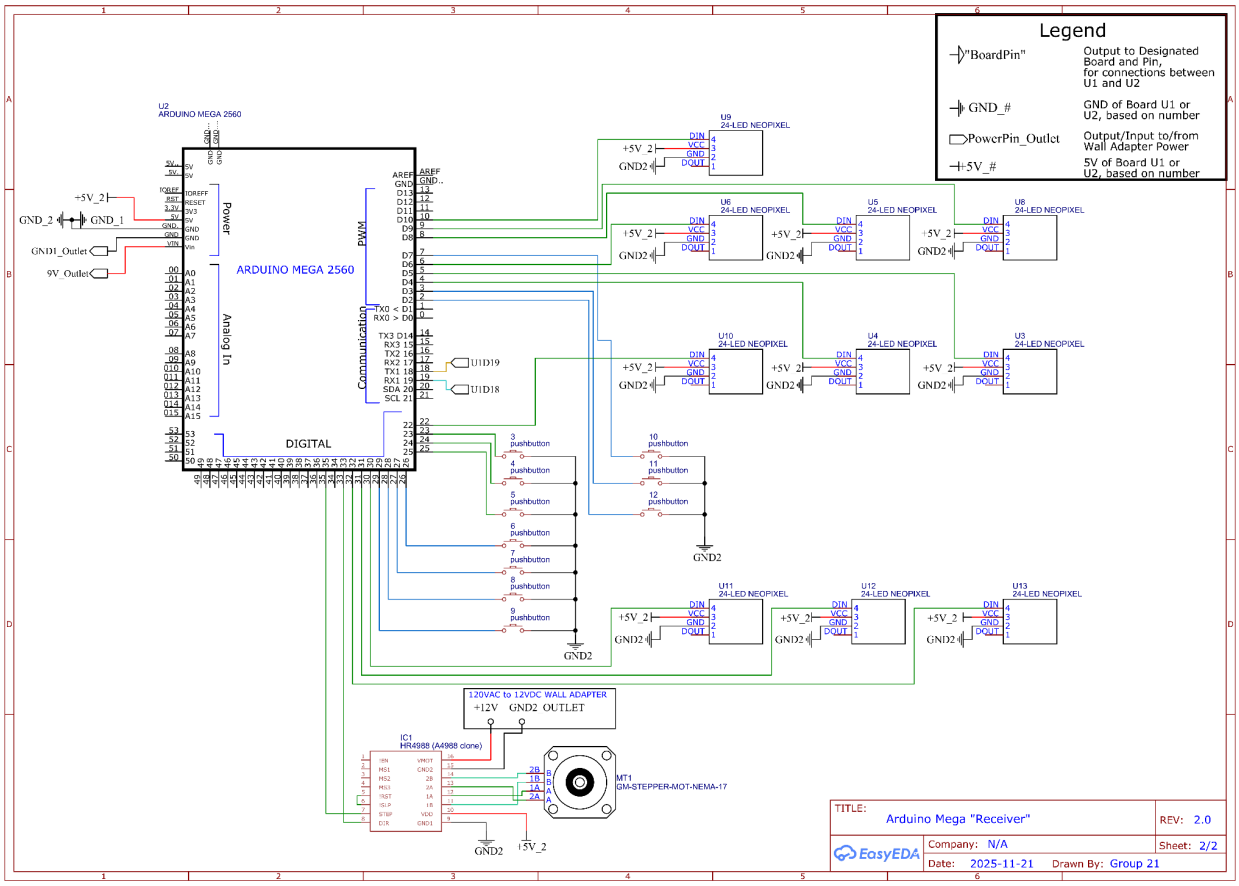


Figure 11: Wiring schematic for the Arduino Mega used for setting difficulty and starting the game, nicknamed “Receiver”.